

# PL-SQL Lab 1

S VIDISH KUMAR

AM.SC.U4AIE23149

```
3 ✓ CREATE TABLE Department (  
4     dept_no INT PRIMARY KEY,  
5     dept_name VARCHAR(100));  
6 ✓ CREATE TABLE Employee (  
7     emp_no INT PRIMARY KEY,  
8     emp_name VARCHAR(100),  
9     emp_sal NUMERIC(10, 2),  
10    dept_no INT REFERENCES Department(dept_no));  
11 -- Sample Data for Testing  
12 ✓ INSERT INTO Department (dept_no, dept_name) VALUES  
13     (10, 'Accounting'),  
14     (20, 'Research');  
15 ✓ INSERT INTO Employee (emp_no, emp_name, emp_sal, dept_no) VALUES  
16     (101, 'Alice', 4000, 10),  
17     (102, 'Bob', 6000, 10), -- Dept 10 Avg = 5000  
18     (103, 'Charlie', 7000, 20),  
19     (104, 'David', 8000, 20); -- Dept 20 Avg = 7500
```

## Question 1.A

```

39 ✓ CREATE OR REPLACE FUNCTION CountEmployeesByDeptNo(department_id INT)
40 RETURNS INT AS $$
41 DECLARE
42     employee_count INT;
43 ✓ BEGIN
44     SELECT COUNT(*)
45     INTO employee_count
46     FROM Employee
47     WHERE dept_no = department_id;
48
49     RETURN employee_count;|
50 END;
51 $$ LANGUAGE plpgsql;
52
53 SELECT CountEmployeesByDeptNo(10);
54

```

Data Output   Messages   Notifications

countemployeesbydeptno  
integer

|   |   |
|---|---|
| 1 | 2 |
|---|---|

## Question 1.C

```

62 ✓ CREATE OR REPLACE PROCEDURE GiveHikeIfBelowDeptAvg(employee_id INT)
63 AS $$
64 DECLARE
65     current_sal NUMERIC;
66     current_dept_no INT;
67     avg_dept_sal NUMERIC;
68 ✓ BEGIN
69     SELECT emp_sal, dept_no
70     INTO current_sal, current_dept_no
71     FROM Employee
72     WHERE emp_no = employee_id;
73 ✓ SELECT AVG(emp_sal)
74     INTO avg_dept_sal
75     FROM Employee
76     WHERE dept_no = current_dept_no;
77 ✓ IF avg_dept_sal > current_sal THEN
78     UPDATE Employee
79     SET emp_sal = emp_sal * 1.10 -- 10% hike
80     WHERE emp_no = employee_id;
81     END IF;
82 END;
83 $$ LANGUAGE plpgsql;
84
85
86 CALL GiveHikeIfBelowDeptAvg(101);
87 SELECT * FROM Employee WHERE emp_no = 101;
88

```

Data Output Messages Notifications

|   | emp_no<br>[PK] integer | emp_name<br>character varying (100) | emp_sal<br>numeric (10,2) | dept_no<br>integer |
|---|------------------------|-------------------------------------|---------------------------|--------------------|
| 1 | 101                    | Alice                               | 4400.00                   | 10                 |

```

89
90 ✓ CREATE OR REPLACE FUNCTION GetDeptNameByEmpName(employee_name VARCHAR)
91 RETURNS VARCHAR AS $$
92 DECLARE
93     department_name VARCHAR;
94 ✓ BEGIN
95     SELECT d.dept_name
96     INTO department_name
97     FROM Employee e
98     JOIN Department d ON e.dept_no = d.dept_no
99     WHERE e.emp_name = employee_name;
100
101     RETURN department_name;
102 END;
103 $$ LANGUAGE plpgsql;
104 SELECT GetDeptNameByEmpName('Alice');
105

```

Data Output Messages Notifications

|   | getdeptnamebyempname<br>character varying |
|---|-------------------------------------------|
| 1 | Accounting                                |

2. Consider the following schema .  
Emp(eno, ename, sal)  
EMP\_PROJ(eno, pno, prjt\_hrs).

- a) Write a function ProjectLoad that returns the total project working hours for the given eno.

2.a

```
23 CREATE OR REPLACE FUNCTION ProjectLoad(employee_no INT)
24 RETURNS NUMERIC AS $$
25 DECLARE
26     total_hours NUMERIC;
27 BEGIN
28     SELECT SUM(prjt_hrs)
29     INTO total_hours
30     FROM EMP_PROJ
31     WHERE eno = employee_no;
32 IF total_hours IS NULL THEN
33     total_hours := 0;
34 END IF;
35
36     RETURN total_hours;
37 END;
38 $$ LANGUAGE plpgsql;
39
40 SELECT ProjectLoad(101);
```

Data Output Messages Notifications

| projectload |    |
|-------------|----|
| numeric     |    |
| 1           | 15 |

- 2.c Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary.

New salary is current sal + difference between current sal and average salary

```

42 --q2c
43 ✓ CREATE OR REPLACE PROCEDURE UpdateLowSalaries()
44 AS $$
45 DECLARE
46     avg_salary NUMERIC;
47 ✓ BEGIN
48     SELECT AVG(sal)
49     INTO avg_salary
50     FROM Emp;
51 ✓ UPDATE Emp
52     SET sal = sal + (avg_salary - sal)
53     WHERE sal < avg_salary;
54 END;
55 $$ LANGUAGE plpgsql;
56
57 CALL UpdateLowSalaries();
58

```

Data Output   Messages   Notifications

CALL

Query returned successfully in 53 msec.

- Consider the following tables  
Item(ino, iname, unit\_price)  
Transaction(tr\_no,ino,qty)

Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

```

104 ✓ CREATE OR REPLACE PROCEDURE CheckAndDeleteItem(item_id INT)
105 AS $$
106 DECLARE
107     transaction_count INT;
108 ✓ BEGIN
109     SELECT COUNT(*)
110     INTO transaction_count
111     FROM Transaction
112     WHERE ino = item_id;
113 ✓ IF transaction_count < 2 THEN
114     DELETE FROM Item
115     WHERE ino = item_id;
116 END IF;
117 END;
118 $$ LANGUAGE plpgsql;
119
120 CALL CheckAndDeleteItem(123);
121

```

Data Output Messages Notifications

CALL

Query returned successfully in 55 msec.

4.

Consider a Bank database which includes the following tables.

ACCOUNTS(ac\_no,br\_no, cust\_no,ac\_type,bal)

BRANCHES(br\_no, br\_name, loc)

CUSTOMER(cno, cname, c\_type)

- a) Write a function that accepts a threshold value and a customer number .  
The program updates the c\_type based on the threshold value. Ie. If balance > threshold then class A, else class B.

**Q4A**



```
Query  Query history
178 ✓ CREATE OR REPLACE PROCEDURE UpdateCustomerType(
179     customer_id INT,
180     threshold_value NUMERIC)AS $$
181 DECLARE
182     total_balance NUMERIC;
183     new_type CHAR(1);
184 ✓ BEGIN
185     SELECT SUM(bal)
186     INTO total_balance
187     FROM ACCOUNTS
188     WHERE cust_no = customer_id;
189 ✓ IF total_balance IS NULL THEN
190     total_balance := 0;
191 END IF;
192 ✓ IF total_balance > threshold_value THEN
193     new_type := 'A';
194 ✓ ELSE
195     new_type := 'B';
196 END IF;
197 ✓ UPDATE CUSTOMER
198 SET c_type = new_type
199 WHERE cno = customer_id;
200 END;
201 $$ LANGUAGE plpgsql;
202
203 CALL UpdateCustomerType(1, 5000.00);
204

Data Output  Messages  Notifications
CALL

Query returned successfully in 75 msec.
```

Q4B) Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.

```

216  CREATE OR REPLACE PROCEDURE CloseBranch(
217      closing_branch_id INT,
218      takeover_branch_id INT
219  )
220  AS $$
221  BEGIN
222      UPDATE ACCOUNTS
223      SET br_no = takeover_branch_id
224      WHERE br_no = closing_branch_id;
225  DELETE FROM BRANCHES
226  WHERE br_no = closing_branch_id;
227  END;
228  $$ LANGUAGE plpgsql;
229  CALL CloseBranch(10, 20);
230

```

Data Output   Messages   Notifications

CALL

Query returned successfully in 76 msec.

**Q4C)**

Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.



```

243 CREATE OR REPLACE FUNCTION SafeWithdrawal(
244     account_id INT,
245     withdrawal_amount NUMERIC
246 )
247 RETURNS TEXT AS $$
248 DECLARE
249     current_balance NUMERIC;
250 BEGIN
251     SELECT bal
252     INTO current_balance
253     FROM ACCOUNTS
254     WHERE ac_no = account_id
255     FOR UPDATE;
256 IF current_balance >= withdrawal_amount THEN
257     UPDATE ACCOUNTS
258     SET bal = bal - withdrawal_amount
259     WHERE ac_no = account_id;
260
261     RETURN 'Withdrawal successful.';
262 ELSE
263     RETURN 'Insufficient funds. Withdrawal denied.';
264 END IF;
265 END;
266 $$ LANGUAGE plpgsql;
267
268 SELECT SafeWithdrawal(1001, 200.00);
269

```

Data Output Messages Notifications



|   |                        |   |
|---|------------------------|---|
|   | safewithdrawal<br>text | 🔒 |
| 1 | Withdrawal successful. |   |